```
import pandas as pd
import gdown
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

```
file_id = '1aONGim4IhTLujjV6WsQhhLNMXtYUy2Rg'
url = f'https://drive.google.com/uc?id={file_id}'
gdown.download(url, 'dataframe-calidad-aire.csv', quiet=False)
```

```
Downloading...
From: https://drive.google.com/uc?id=1aONGim4IhTLujjV6WsQhhLNMXtYUy2Rg
To: /content/dataframe-calidad-aire.csv
100%|██████████| 11.8M/11.8M [00:00<00:00, 107MB/s]
'dataframe-calidad-aire.csv'
```

```
df = pd.read_csv('dataframe-calidad-aire.csv')
df.head()
```

	FECHA	VELOCIDAD_VIENTO	DIRECCION_VIENTO	PRECIPITACION	CO	SO2	NO2	OZONO	PM10	PM2.5	
0	2021-01-01 01:00:00	0.0	333.0	0.0	0.64	0.2	16.4	1.0	21.6	21.6	CENTRO_D
1	2021-01-01 02:00:00	0.3	263.0	0.0	0.77	0.3	14.6	0.7	30.9	25.0	CENTRO_D
2	2021-01-01 05:00:00	0.3	350.0	0.0	0.79	0.4	11.0	0.9	52.5	52.5	CENTRO_D
3	2021-01-01 06:00:00	0.3	340.0	0.0	0.67	0.4	11.3	1.2	50.0	49.0	CENTRO_D
4	2021-01-01 07:00:00	0.1	211.0	0.0	0.47	0.4	12.0	2.1	51.4	37.0	CENTRO_D

```
df
```

	FECHA	VELOCIDAD_VIENTO	DIRECCION_VIENTO	PRECIPITACION	CO	SO2	NO2	OZONO	PM10	PM2.5	
0	2021-01-01 01:00:00	0.0	333.0	0.0	0.64	0.2	16.4	1.0	21.6	21.6	CENT
1	2021-01-01 02:00:00	0.3	263.0	0.0	0.77	0.3	14.6	0.7	30.9	25.0	CENT
2	2021-01-01 05:00:00	0.3	350.0	0.0	0.79	0.4	11.0	0.9	52.5	52.5	CENT
3	2021-01-01 06:00:00	0.3	340.0	0.0	0.67	0.4	11.3	1.2	50.0	49.0	CENT
4	2021-01-01 07:00:00	0.1	211.0	0.0	0.47	0.4	12.0	2.1	51.4	37.0	CENT
...	...	...	...	...	...	...	...	...	...	...	
135082	NaN	0.9	160.0	0.0	0.29	1.1	3.7	23.4	42.2	10.0	
135083	NaN	0.8	217.0	0.0	0.28	1.0	4.3	22.5	60.5	10.0	
135084	NaN	2.5	112.0	0.0	0.33	1.1	4.3	22.0	44.4	12.0	
135085	NaN	3.6	121.0	0.0	0.29	1.1	3.5	22.4	14.9	5.0	
135086	NaN	3.9	110.0	0.0	0.28	0.9	2.9	20.8	11.2	1.0	

135087 rows × 13 columns

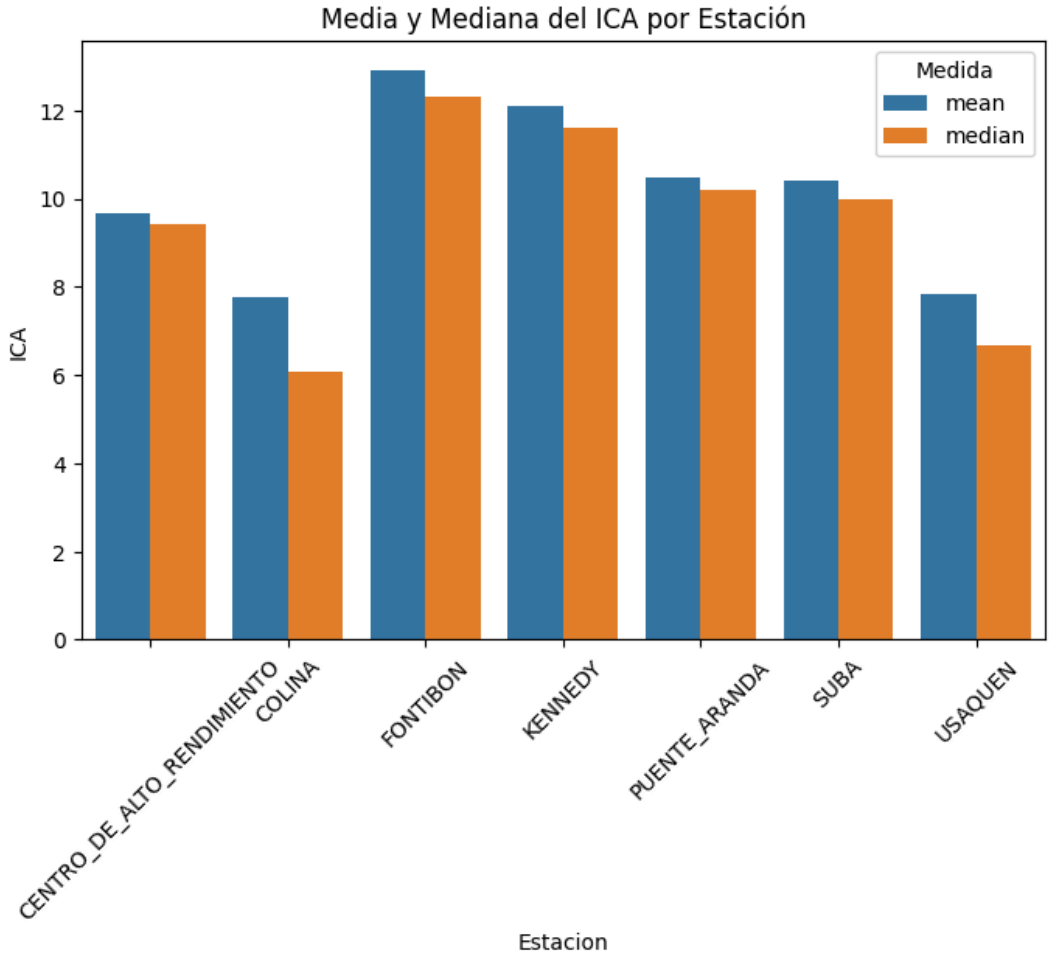
```
df.describe()
```

	VELOCIDAD_VIENTO	DIRECCION_VIENTO	PRECIPITACION	CO	SO2	NO2	
count	135087.000000	135087.000000	135087.000000	135087.000000	135087.000000	135087.000000	135087
mean	1.588416	183.892440	0.122346	0.517857	1.201466	15.377777	12
std	1.091673	104.051392	0.965598	0.370584	1.390478	8.835573	11
min	0.000000	0.000000	0.000000	-0.040000	0.000000	0.000000	0
25%	0.800000	102.000000	0.000000	0.300000	0.600000	8.600000	2
50%	1.300000	182.000000	0.000000	0.420000	0.900000	14.500000	9
75%	2.100000	276.000000	0.000000	0.620000	1.400000	20.700000	18
max	8.300000	360.000000	52.400000	12.030000	42.000000	98.300000	106

```
# Agrupamos por estación y calculamos media y mediana
df_agrupado = df.groupby('Estacion')['ICA'].agg(['mean', 'median']).reset_index()

# --- Gráfico comparativo ---
plt.figure(figsize=(8,5))
df_melt = df_agrupado.melt(
    id_vars='Estacion', value_vars=['mean','median'],
    var_name='Medida', value_name='ICA'
)
sns.barplot(data=df_melt, x='Estacion', y='ICA', hue='Medida')
```

```
plt.title("Media y Mediana del ICA por Estación")
plt.xticks(rotation=45)
plt.show()
```




```
def indice_asimetria_pearson(serie):
    media = serie.mean()
    mediana = serie.median()
    std = serie.std()

    if std == 0:
        return 0  # evita división por cero

    return (media - mediana) / std

# Agrupar por estación y calcular el índice de asimetría
df_asimetria = df.groupby("Estacion")["ICA"].apply(indice_asimetria_pearson).reset_index()
df_asimetria.columns = ["Estacion", "Indice_Asimetria_Pearson"]

print(df_asimetria)
```

```
                     Estacion  Indice_Asimetria_Pearson
0  CENTRO_DE_ALTO_RENDIMIENTO                  0.047291
1                      COLINA                  0.353269
2                    FONTIBON                  0.108651
3                     KENNEDY                  0.091118
4               PUENTE_ARANDA                  0.047026
5                        SUBA                  0.076969
6                     USAQUEN                  0.248054
```

La diferencia entre media y mediana es relativamente baja en la mayoría de estaciones, lo que sugiere que no hay fuerte sesgo o asimetría general en los datos. Esto indica que los valores extremos no están afectando significativamente la tendencia central de los datos en general.

Sin embargo, la desviación estándar (STD) sigue siendo más del 50% de la media en casi todas las estaciones, lo que implica alta variabilidad: hay momentos con ICA mucho más alto que otros. Esta dispersión sugiere que la calidad del aire no es constante y puede verse afectada por factores puntuales como clima, tráfico o actividades industriales.

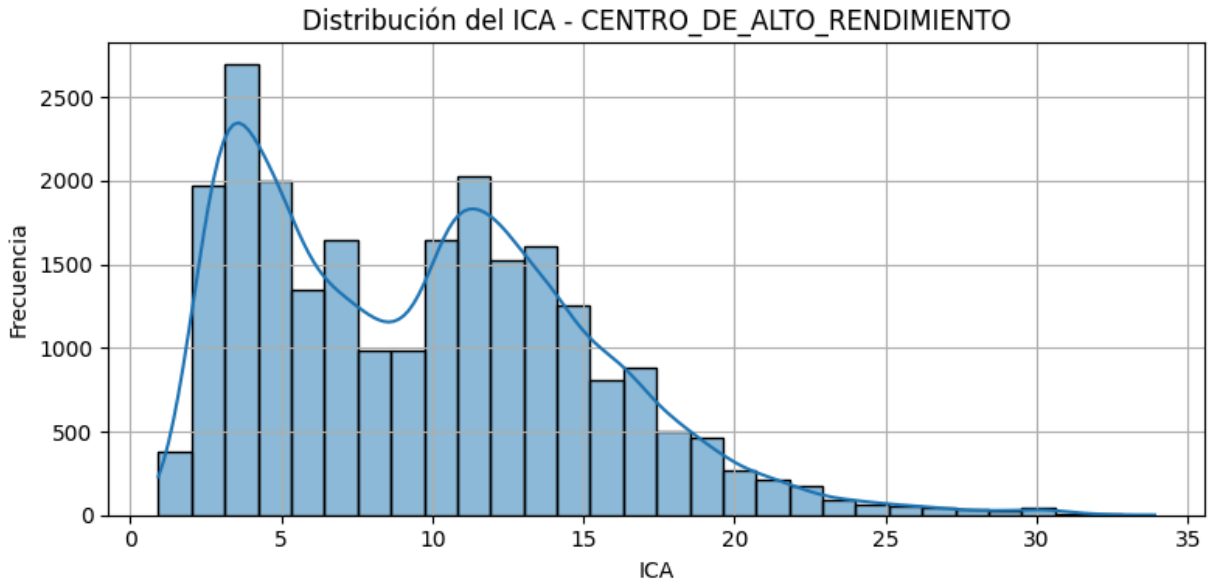
FONTIBON, KENNEDY, PUENTE_ARANDA, SUBA: Estaciones con ICA promedio más alto. Estas estaciones probablemente están influenciadas por actividad industrial y/o tráfico denso, así como por condiciones locales desfavorables para la dispersión de contaminantes como vientos débiles o una topografía cerrada.

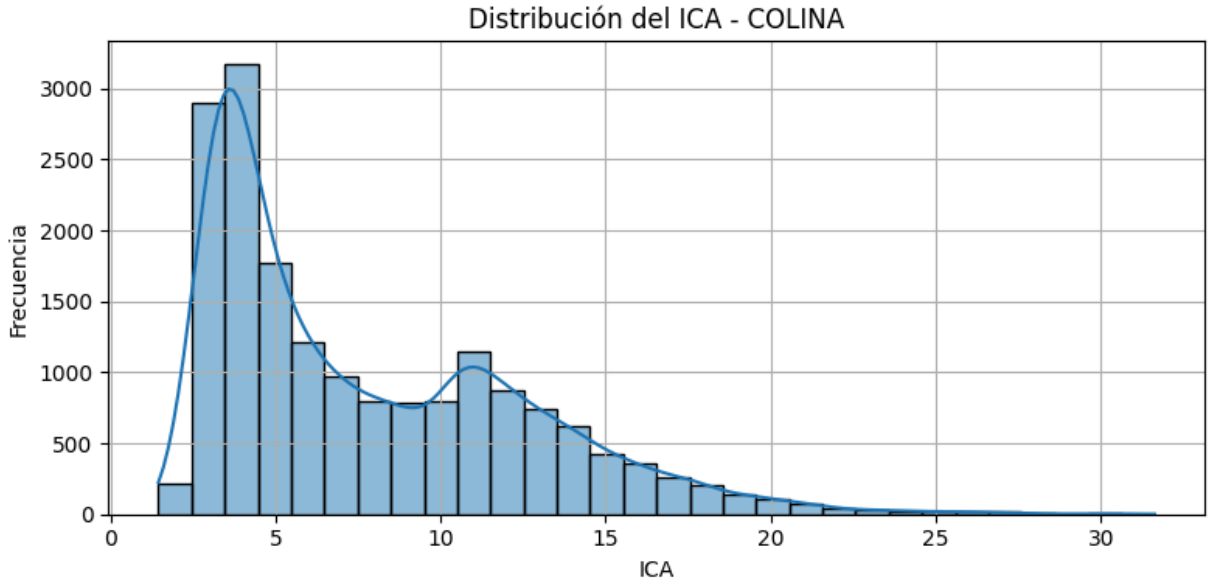
COLINA, USAQUEN: Estaciones con ICA más bajo. Estas pueden tener mejores condiciones de ventilación (zonas más altas, con menos tráfico o actividad industrial). También podrían beneficiarse de mayor cobertura vegetal o menor densidad urbana, lo que ayuda a mantener una mejor calidad del aire.

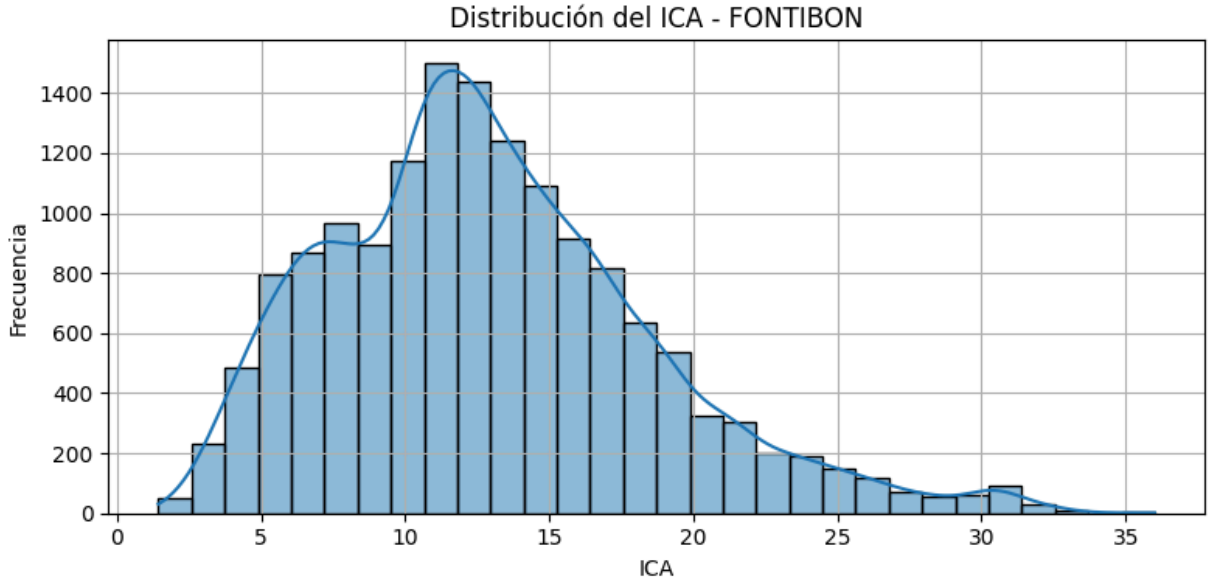
PUENTE_ARANDA, FONTIBON, KENNEDY: Alta dispersión (STD) indica que la calidad del aire fluctúa mucho durante el periodo analizado. Esto puede deberse a variabilidad horaria significativa por tráfico, condiciones meteorológicas o emisiones intermitentes de fuentes puntuales como fábricas o eventos especiales.

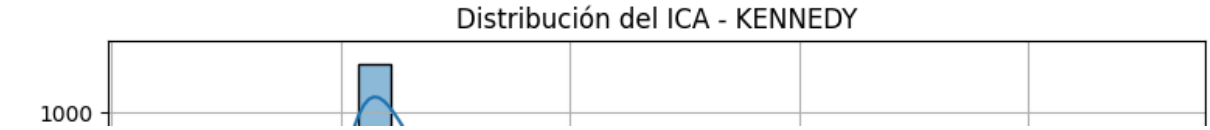
COLINA, USAQUEN: Diferencia significativa entre media y mediana. Esto puede indicar una distribución sesgada a la derecha, es decir, usualmente hay ICA bajos, pero hay eventos ocasionales de contaminación alta que elevan la media. Esto sugiere que aunque la calidad del aire suele ser buena, no está exenta de episodios críticos.

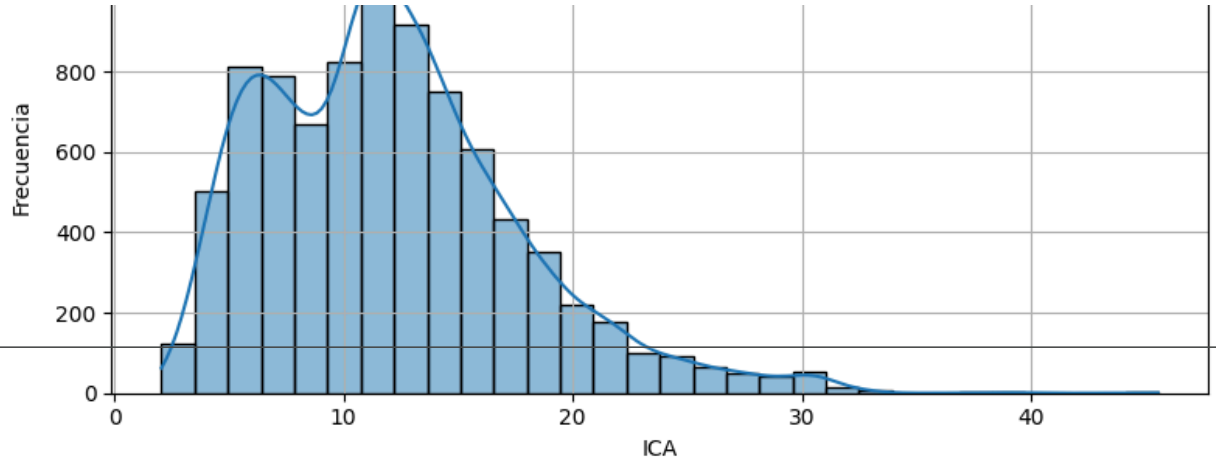
```
# Histograma + KDE por estación
for estacion, datos in df_agrupado:
    plt.figure(figsize=(8, 4))
    sns.histplot(datos['ICA'], bins=30, kde=True)
    plt.title(f'Distribución del ICA - {estacion}')
    plt.xlabel('ICA')
    plt.ylabel('Frecuencia')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

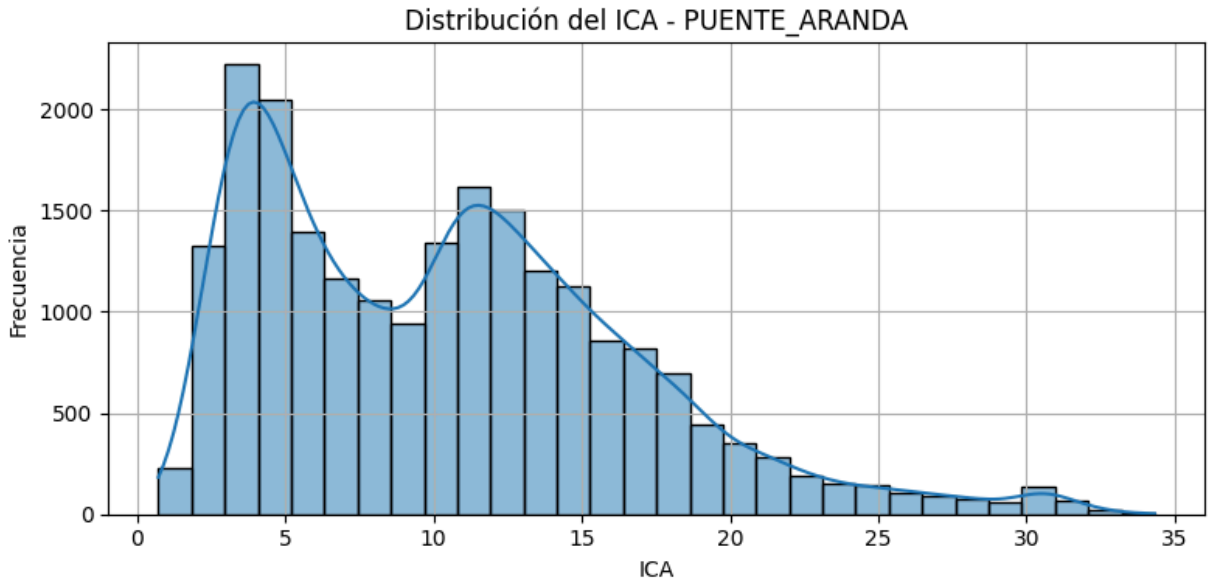

Distribución del ICA - CENTRO_DE_ALTO_RENDIMIENTO
Frecuencia
ICA

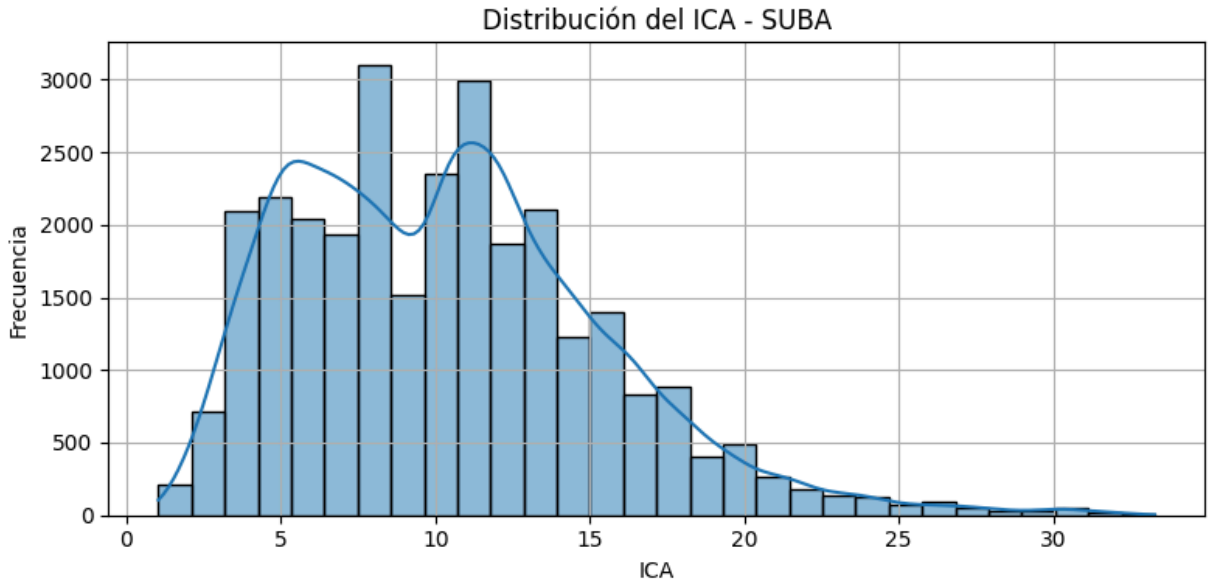

Distribución del ICA - COLINA
Frecuencia
ICA

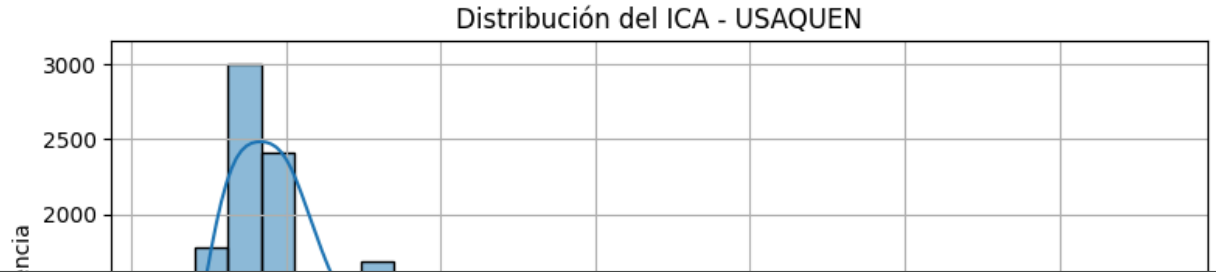

Distribución del ICA - FONTIBON
Frecuencia
ICA


Distribución del ICA - KENNEDY

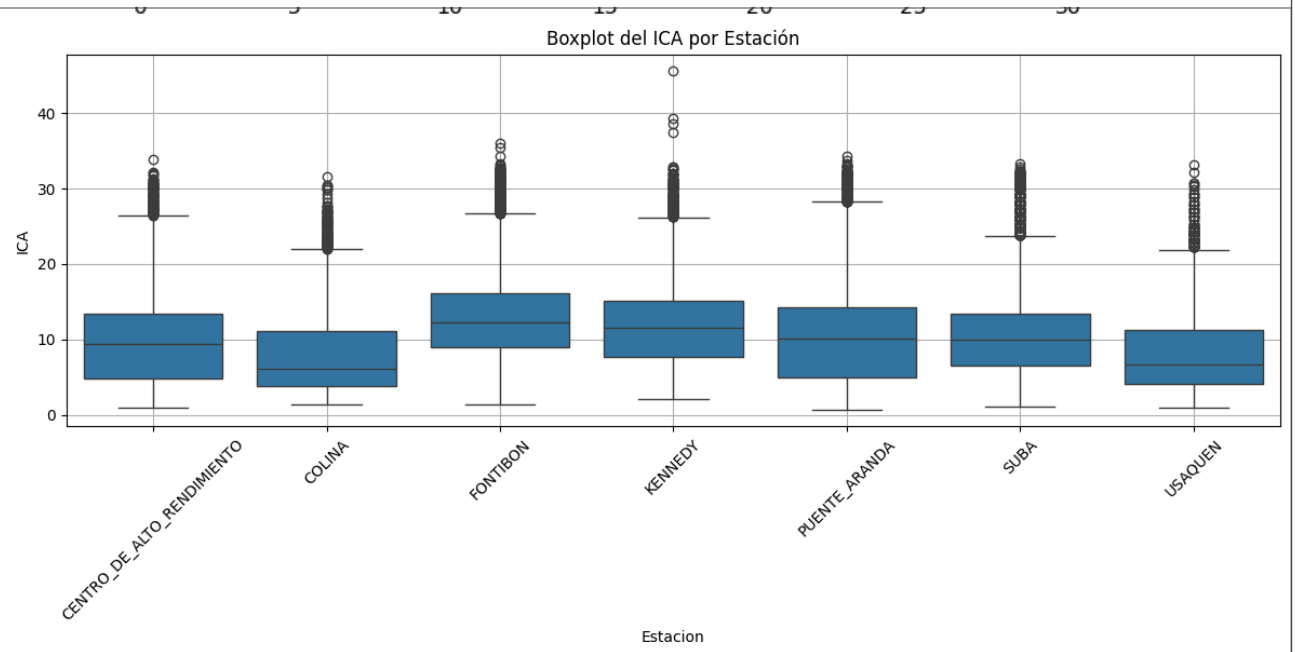

Frecuencia
ICA


Distribución del ICA - PUENTE_ARANDA
Frecuencia
ICA


Distribución del ICA - SUBA
Frecuencia
ICA


Distribución del ICA - USAQUEN

```
# Boxplot por estación
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='Estacion', y='ICA')
plt.title('Boxplot del ICA por Estación')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

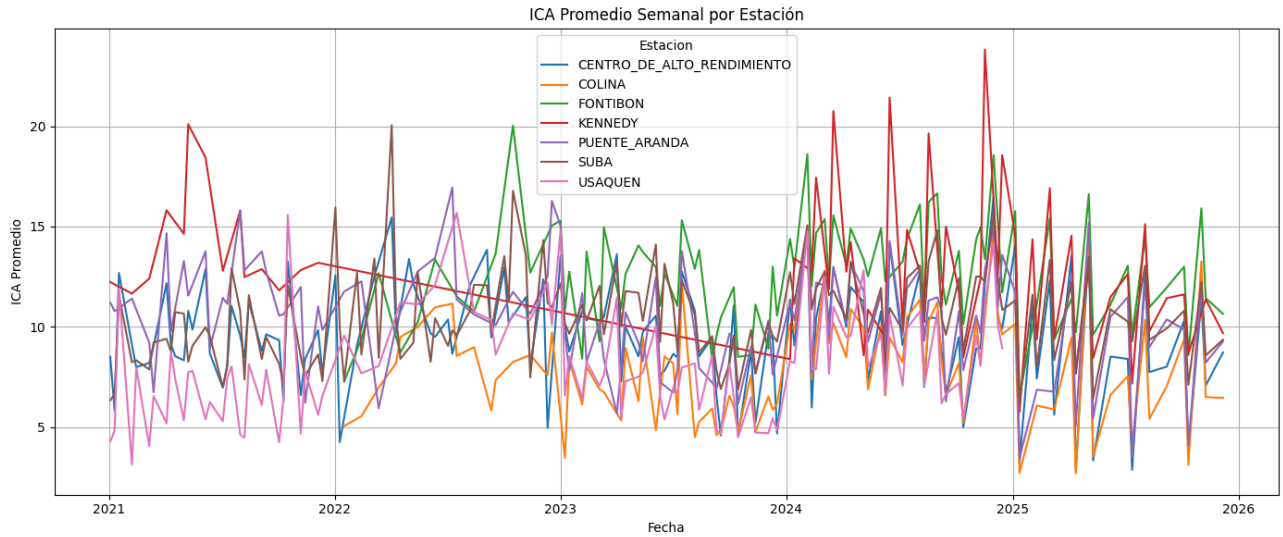


El boxplot del ICA por estación permite observar diferencias importantes en la calidad del aire entre distintas zonas. En general, las medianas son similares, lo que indica que el nivel típico de calidad del aire no varía drásticamente entre estaciones. Sin embargo, algunas estaciones como Fontibón, Kennedy y Puente Aranda presentan valores más altos tanto en la mediana como en el rango intercuartílico, lo que sugiere que en estas zonas la calidad del aire tiende a ser peor y más variable. Por el contrario, estaciones como Colina y Usaquén muestran cajas más bajas, lo que indica mejor calidad del aire y menor variabilidad. En todas las estaciones se observan outliers, es decir, eventos de contaminación mucho más altos que el promedio, pero Kennedy destaca por tener los valores atípicos más extremos. En resumen, el boxplot respalda que ciertas estaciones enfrentan una mayor frecuencia y severidad de eventos de mala calidad del aire, mientras que otras mantienen condiciones más estables y favorables.

```
# Distribución temporal del ICA por estación
df['FECHA'] = pd.to_datetime(df['FECHA'], errors='coerce')
df_semanal = df.set_index('FECHA').groupby('Estacion')['ICA'].resample('W').mean().reset_index()

plt.figure(figsize=(14, 6))
sns.lineplot(data=df_semanal, x='FECHA', y='ICA', hue='Estacion')
plt.title('ICA Promedio Semanal por Estación')
plt.xlabel('Fecha')
plt.ylabel('ICA Promedio')
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



El gráfico muestra la evolución del ICA promedio semanal en distintas estaciones de monitoreo de calidad del aire entre los años 2021 y 2026. Se observan fluctuaciones importantes en todas las estaciones, con picos esporádicos que indican episodios de contaminación más severos. Estaciones como Kennedy, Fontibón y Puente Aranda tienden a registrar valores más altos de ICA con mayor frecuencia, lo que sugiere que estas zonas experimentan condiciones de calidad del aire más críticas de manera recurrente.

En contraste, Colina, Usaquén y Centro de Alto Rendimiento suelen tener valores más bajos y estables, lo que podría reflejar mejores condiciones ambientales o menor influencia de fuentes contaminantes.

Aunque hay bastante variabilidad interanual, la tendencia general parece mantenerse sin un deterioro significativo a largo plazo, e incluso con una leve disminución en algunas estaciones. Sin embargo, la densidad de picos recientes sugiere que los eventos de contaminación son cada vez más frecuentes o intensos en ciertos periodos, lo cual debería investigarse más a fondo.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten

# 1. Filtra columnas relevantes
features = ['VELOCIDAD_VIENTO', 'DIRECCION_VIENTO', 'PRECIPITACION',
            'CO', 'SO2', 'NO2', 'OZONO', 'PM10', 'PM2.5', 'ICA']
target_col = 'ICA'

df = df.sort_values('FECHA')
```

```
df = df[features].copy()
df = df.dropna().reset_index(drop=True)

# 2. Crea ventanas de 24h con todas las features
def create_multivariate_sequences(data, target_index, window_size=24):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i + window_size])
        y.append(data[i + window_size, target_index])
    return np.array(X), np.array(y)

# Normaliza cada set por separado para evitar data leakage
# Primero, crea X_raw y y_raw (sin escalar)
X_raw, y_raw = create_multivariate_sequences(df.values, target_index=features.index(target_col), window_siz

# Divide en train/val/test antes de escalar
n = len(X_raw)
train_end = int(n * 0.6)
val_end = int(n * 0.8)

X_train_raw, y_train = X_raw[:train_end], y_raw[:train_end]
X_val_raw, y_val = X_raw[train_end:val_end], y_raw[train_end:val_end]
X_test_raw, y_test = X_raw[val_end:], y_raw[val_end:]

# Escalar usando solo entrenamiento
scaler = StandardScaler()
X_train_reshaped = X_train_raw.reshape(-1, len(features))
scaler.fit(X_train_reshaped)

# Escala cada set
X_train = scaler.transform(X_train_raw.reshape(-1, len(features))).reshape(X_train_raw.shape)
X_val = scaler.transform(X_val_raw.reshape(-1, len(features))).reshape(X_val_raw.shape)
X_test = scaler.transform(X_test_raw.reshape(-1, len(features))).reshape(X_test_raw.shape)

# Escala también y (solo ICA) por separado
ica_index = features.index(target_col)
ica_mean = scaler.mean_[ica_index]
ica_std = scaler.scale_[ica_index]

y_train = (y_train - ica_mean) / ica_std
y_val = (y_val - ica_mean) / ica_std
y_test = (y_test - ica_mean) / ica_std

# 3. Modelo en Keras (Regresión Lineal Multivariable)
model = Sequential([
    Input(shape=(3, len(features))),
    Flatten(),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=10,
    batch_size=32,
    verbose=1
)

# 4. Predicción y desnormalización
y_pred = model.predict(X_test).flatten()
y_pred_denorm = y_pred * ica_std + ica_mean
y_test_denorm = y_test * ica_std + ica_mean

# 5. Gráfico de resultados
plt.figure(figsize=(12, 5))
```

```
plt.plot(y_test_denorm, label='ICA Real', linewidth=1.5)
plt.plot(y_pred_denorm, label='ICA Predicho', linewidth=1.5)
plt.title('Predicción del ICA con 24h de Datos Multivariables')
plt.xlabel('Tiempo (h)')
plt.ylabel('ICA')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 6. Gráfico de pérdida
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Pérdida Entrenamiento')
plt.plot(history.history['val_loss'], label='Pérdida Validación')
plt.title('Evolución de la Pérdida durante el Entrenamiento')
plt.xlabel('Épocas')
plt.ylabel('Pérdida (MSE)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Mostrar el resultado oculto

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Cálculo de métricas sobre los datos desnormalizados
mae = mean_absolute_error(y_test_denorm, y_pred_denorm)
mse = mean_squared_error(y_test_denorm, y_pred_denorm)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_denorm, y_pred_denorm)

# 2. Mostrar resultados
print("Evaluación del Modelo en el Set de Prueba (Desnormalizado):")
print(f"MAE  (Error Absoluto Medio): {mae:.2f}")
print(f"MSE  (Error Cuadrático Medio): {mse:.2f}")
print(f"RMSE (Raíz del Error Cuadrático Medio): {rmse:.2f}")
print(f"R²   (Coeficiente de Determinación): {r2:.4f}")
```

```
Evaluación del Modelo en el Set de Prueba (Desnormalizado):
MAE  (Error Absoluto Medio): 2.33
MSE  (Error Cuadrático Medio): 9.68
RMSE (Raíz del Error Cuadrático Medio): 3.11
R²   (Coeficiente de Determinación): 0.6391
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam

# 1. Selección de features
features = ['VELOCIDAD_VIENTO', 'DIRECCION_VIENTO', 'PRECIPITACION',
            'CO', 'SO2', 'NO2', 'OZONO', 'PM10', 'PM2.5', 'ICA']
target_col = 'ICA'

# 2. Ordena por fecha, filtra y limpia
df = df.sort_values('FECHA')
df = df[features].dropna().reset_index(drop=True)
```